

The background of the image is a Minecraft landscape. It features rolling green hills with some brown patches, a blue river winding through the terrain, and several grey, rocky mountains in the distance. The sky is a clear blue with a few white, blocky clouds. A bright sun is visible in the upper center of the sky. The title "MINECRAFT" is written in large, brown, pixelated letters with a black outline, positioned in the upper middle of the image. Below it, the words "BIOME" and "CLASSIFICATION" are written in smaller, brown, pixelated letters, also with a black outline.

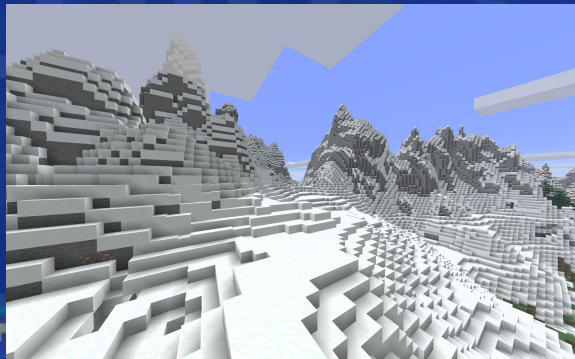
MINECRAFT

BIOME CLASSIFICATION

By Carla Lopez, Sara Zangrilli,
Sebastian Canakis, and Chelsea Malach

Goals:

- Create and compare four different models that classify Minecraft screenshots into their designated biomes.
- The models will distinguish between features such as color, texture, and shapes/edges
- Each model demonstrates a different machine learning concept
 - **Random Forest:** Bagging
 - **SVM:** Margin-based classification
 - **Adaboost:** Sequential Boosting
 - **CNN:** Hierarchical feature learning



Predictions

- CNN will significantly outperform the other classifiers
 - Random Forest
 - SVM
 - AdaBoost

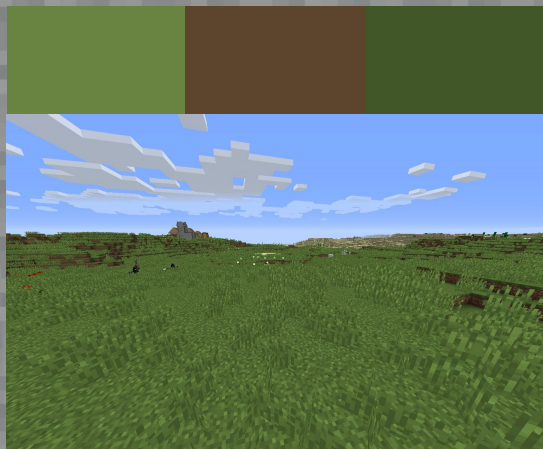
Tools

- **Dataset:** Kaggle
- **Model Training:** Scikit-Learn
- **Data Processing:** NumPy & Pandas
- **Feature Engineering:** Scikit-image and OpenCV
- **Graphs and Plots:** Matplotlib



Dataset

- We found a dataset on Kaggle containing screenshots across 30+ biomes.
- The amount of photos per biome was highly imbalanced, with the image count ranging from over 3000 to 24
 - To prevent class imbalance, we trimmed all the datasets down to the size of our smallest selected set, with 700 images.
- Minecraft has approximately 54 overworld biomes, a lot of which are visually very similar.
- To reduce complexity and improve model clarity we limited the scope to 6 distinct biomes.



Preprocessing

- Images are loaded from individual biome folders and labeled with their biome name.
- Each image was filtered using two quality checks:
 - Color variance threshold
 - Image entropy threshold
- After filtering, up to 600 images are randomly selected per biome to balance the dataset.
- The images are then shuffled and split into 70% Training, 15% Test, and 15% Validation.

HOG

- Captures the shape and structure of terrain in images
- Measures edge directions and gradients
- Effective for distinguishing horizon textures such as:
 - **Mountains:** Sharp, pointed edges
 - **Plains:** Smooth gradients
 - **Dark oak forest:** Dense texture patterns
- This is able to improve accuracy for biomes that might have overall similar color histograms

Color Histogram

- Captures color distribution per image, computed separately for red, green, and blue channels
- Each channel is split into 256 intensity bins
- Effective for distinguishing biome color patterns such as:
 - **Desert:** Warm tones
 - **Plains:** Uniform green tones
 - **Swamp:** Varied green tones.
- This is able to improve accuracy for biomes that have overall similar textures or structures

Feature engineering

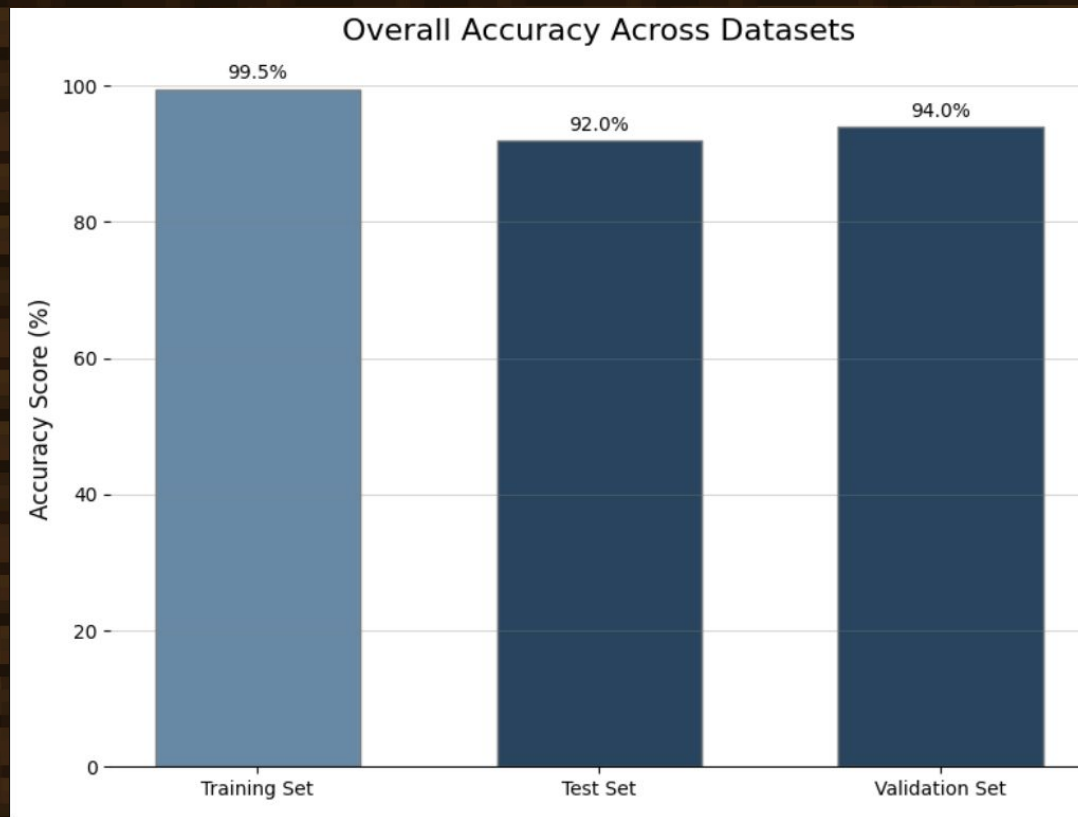
- For Random Forest, SVM, and Adaboost
 - Images are converted into numerical feature vectors using
 - **HOG** features for edge structure and texture
 - **Color histograms** for color distribution
 - Each image is represented as a single combined feature vector
 - Features are extracted for training, validation, and test sets,

Random Forest

- Trained using HOG and Color Histogram
- Model Hyperparameter tuning:
 - Used Scikit-Learn Grid Search for Hyperparameter tuning
 - Used the validation set for this process
- Training
 - Trained the Random Forest Classifier on the processed training set using the optimized hyperparameters found during grid search

```
{  
    "max_depth": null,  
    "max_features": "sqrt",  
    "min_samples_leaf": 2,  
    "min_samples_split": 2,  
    "n_estimators": 100  
}
```


Random Forest Accuracy



Classification Report

Testing Set

Test Set Classification Report:

	precision	recall	f1-score	support
plains	0.98	0.93	0.95	94
desert	0.98	0.98	0.98	89
mountains	0.94	0.95	0.95	84
swamp	0.91	0.91	0.91	82
dark_forest	0.88	0.93	0.91	106
savanna	0.96	0.94	0.95	84
accuracy			0.94	539
macro avg	0.94	0.94	0.94	539
weighted avg	0.94	0.94	0.94	539

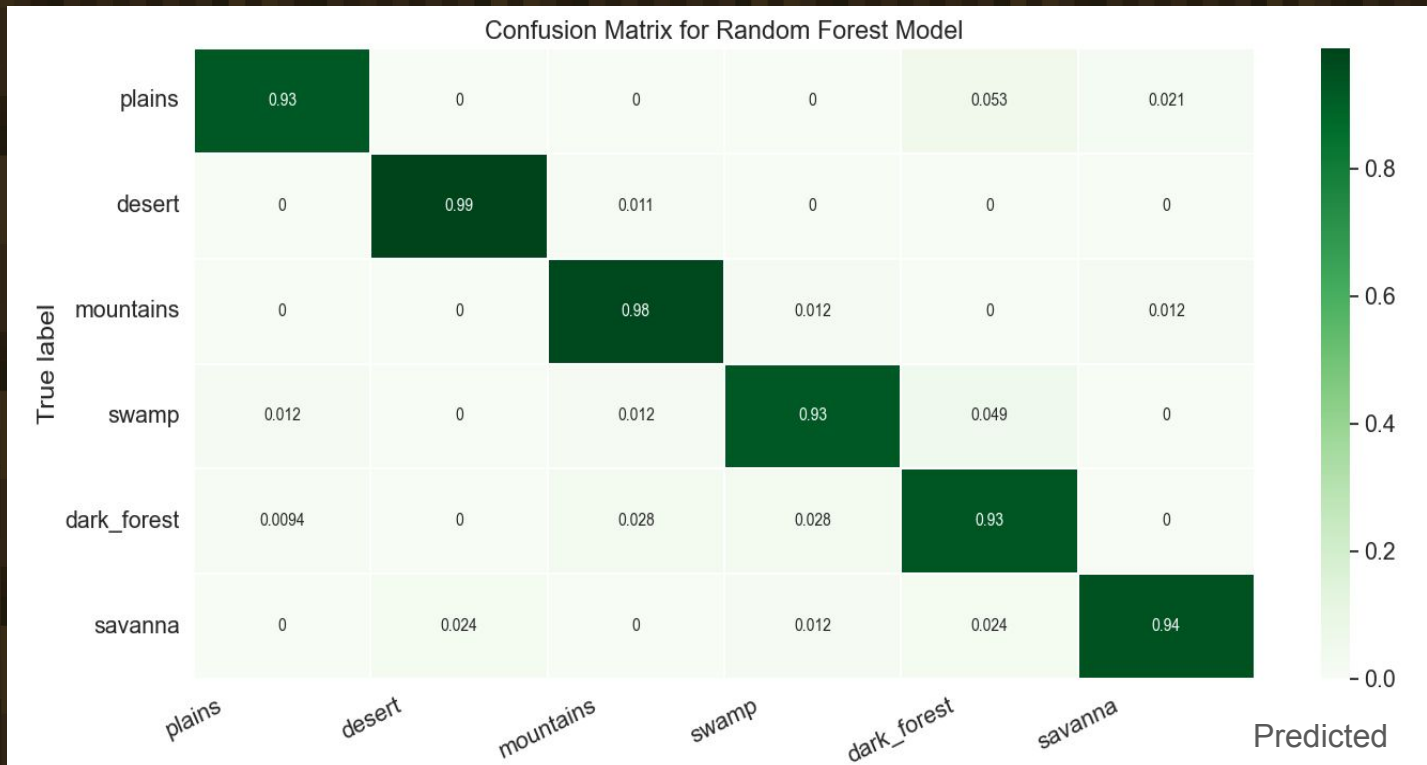
Training Set

Training Set Classification Report:

	precision	recall	f1-score	support
plains	0.99	0.99	0.99	413
desert	0.99	1.00	0.99	432
mountains	1.00	1.00	1.00	411
swamp	1.00	0.99	1.00	425
dark_forest	1.00	1.00	1.00	420
savanna	1.00	0.99	1.00	420
accuracy			1.00	2521
macro avg	1.00	1.00	1.00	2521
weighted avg	1.00	1.00	1.00	2521

Confusion Matrix

Test Set

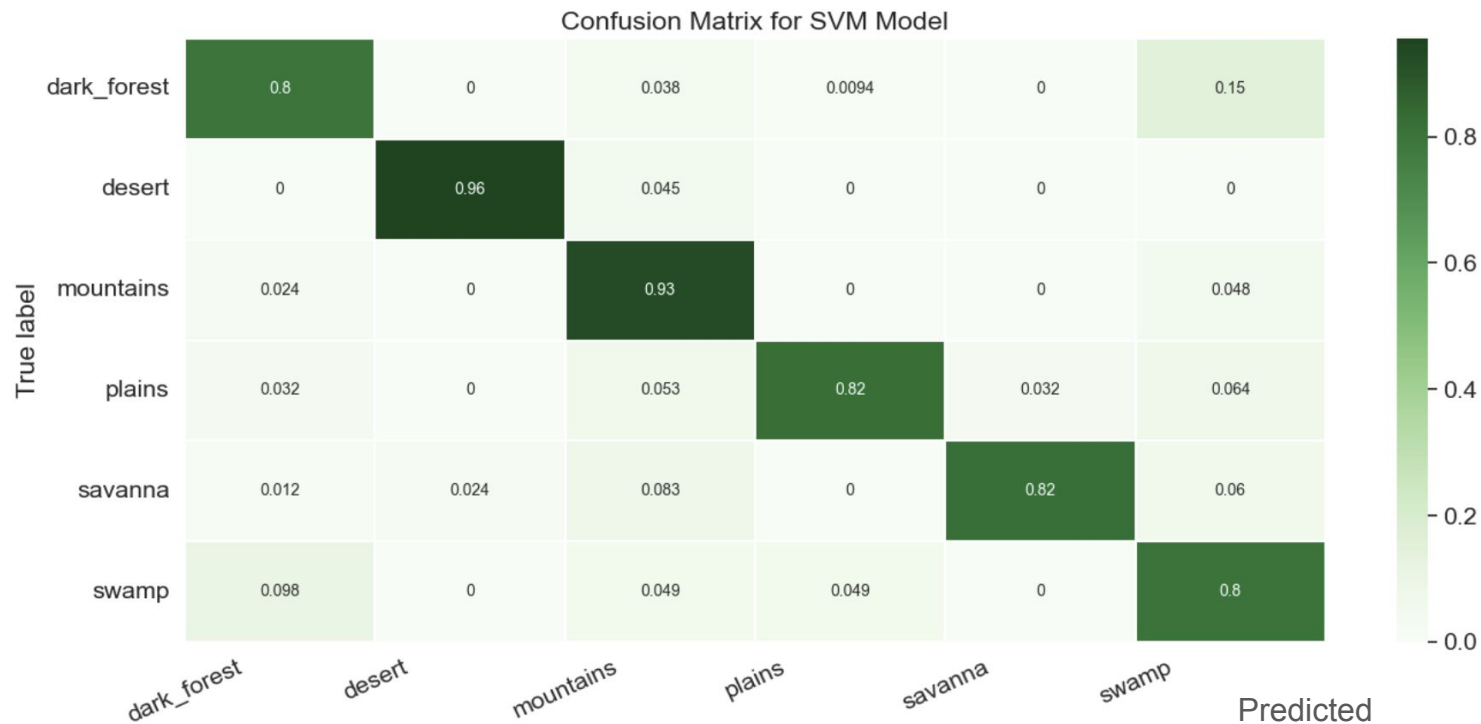


Support Vector Machine

SVM model trained with individual pixels:

- No spatial understanding, sensitive to small changes
- 85% accuracy
- Confused dark forest, plains, swamp, and savanna most often
 - All green
 - Does not take into account shapes/textures of trees, hills, etc

Support Vector Machine



Support Vector Machine

SVM model trained with HOG and color histogram

- HOG: captures information about the shapes and edges
- Color histogram: captures the color distribution of the image
- 93.3% accuracy

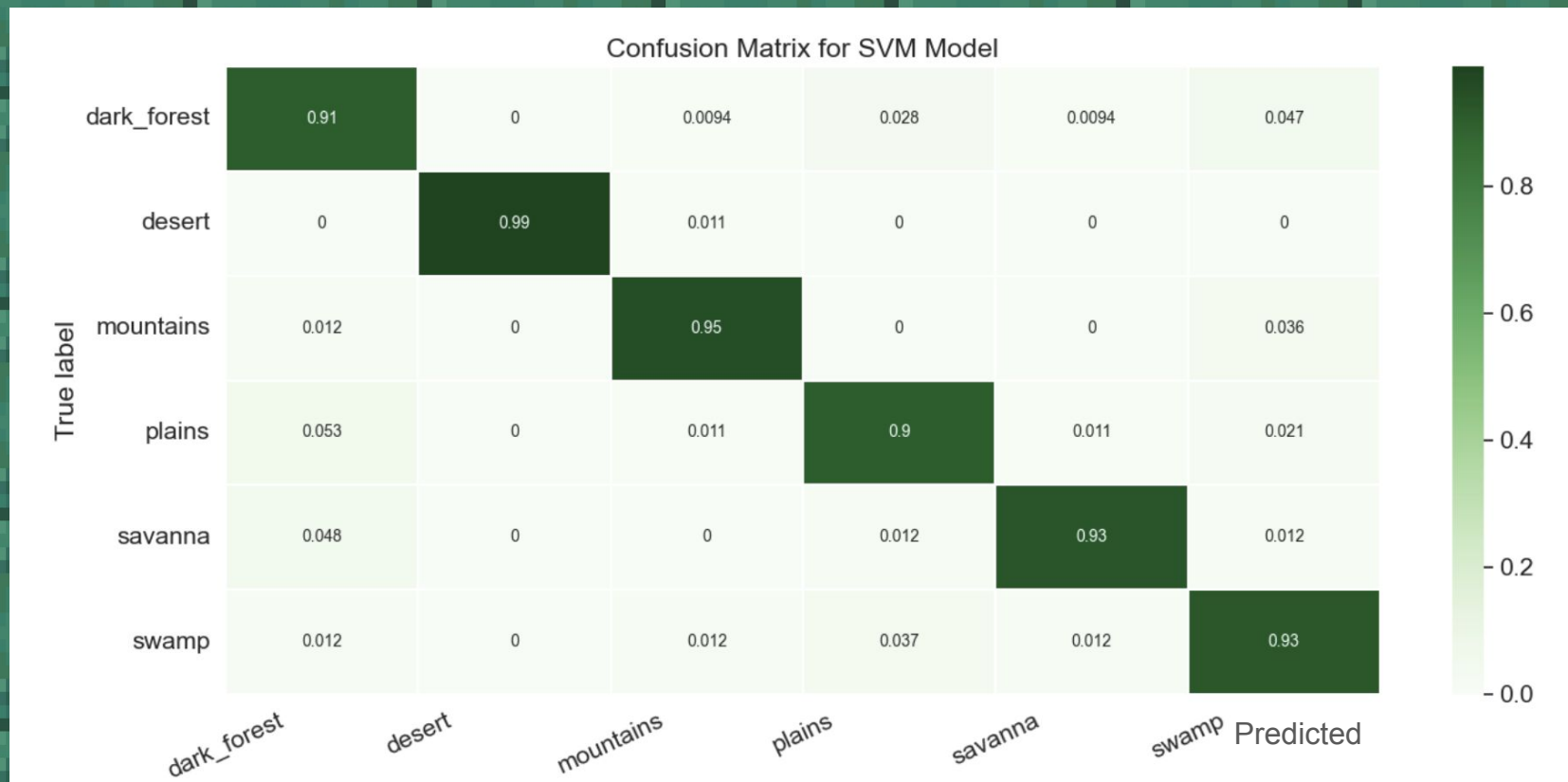
Model parameters:

- Kernel: rbf
- C: 10

Classification Report:

	precision	recall	f1-score	support
dark_forest	0.90	0.91	0.90	106
desert	1.00	0.99	0.99	89
mountains	0.95	0.95	0.95	84
plains	0.92	0.90	0.91	94
savanna	0.96	0.93	0.95	84
swamp	0.87	0.93	0.90	82
accuracy			0.93	539
macro avg	0.94	0.93	0.93	539
weighted avg	0.93	0.93	0.93	539

SVM Confusion Matrix



AdaBoost†

- Model Architecture
 - Implemented using Scikit-Learn AdaBoostClassifier
 - Uses decision stumps combined into a strong classifier
 - Each new learner focuses on previously misclassified examples
- Training:
 - Trained on extracted feature vectors from the training set
 - Model is built sequentially through boosting
 - Final prediction combines weak learners, with higher weight given to more accurate models

AdaBoost Accuracy & classification report

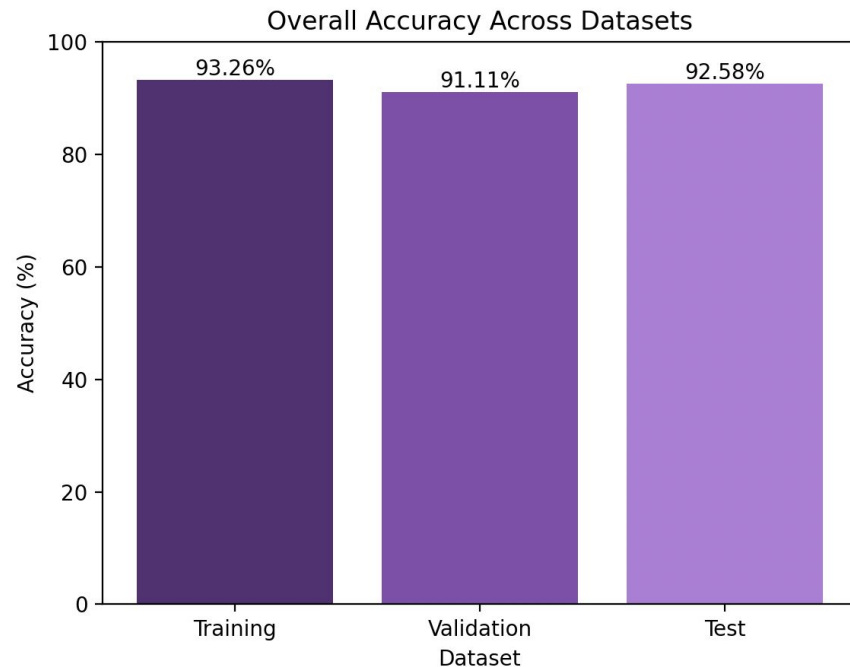
Training Accuracy: 0.9326

Validation Accuracy: 0.9111

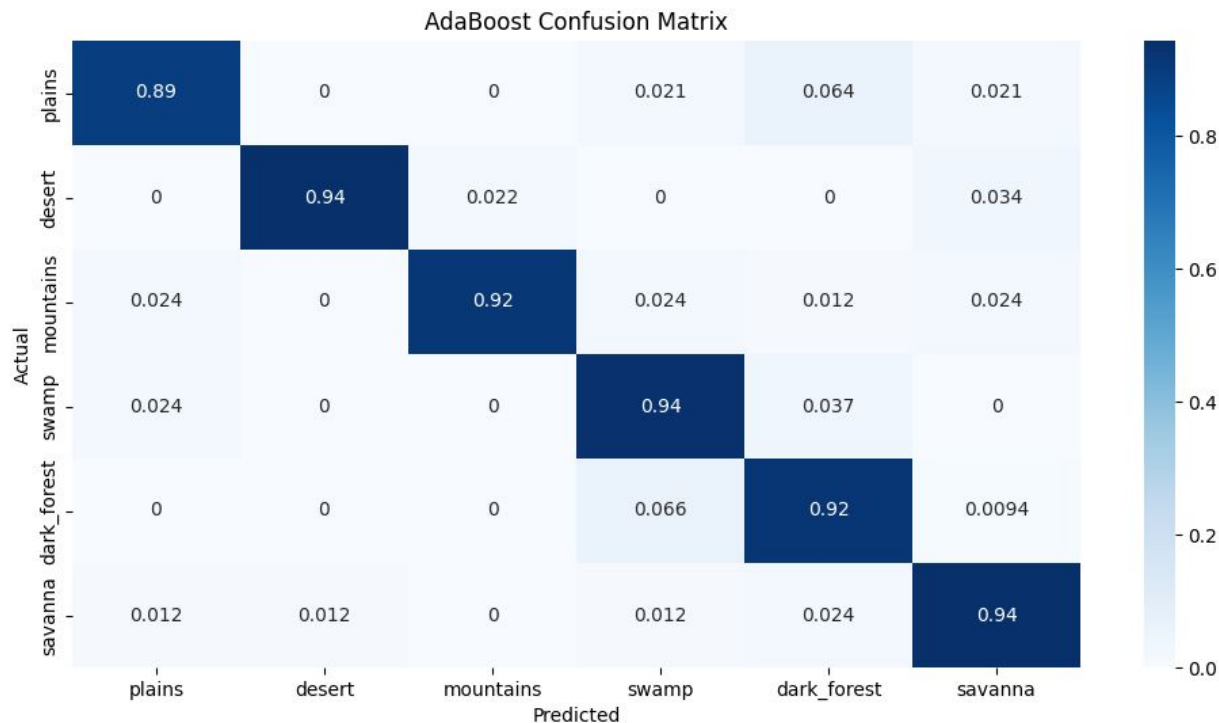
Test Accuracy: 0.9258

Classification Report (Test):

	precision	recall	f1-score	support
plains	0.94	0.89	0.92	94
desert	0.99	0.94	0.97	89
mountains	0.97	0.92	0.94	84
swamp	0.87	0.94	0.90	82
dark_forest	0.89	0.92	0.91	106
savanna	0.91	0.94	0.92	84
accuracy			0.93	539
macro avg	0.93	0.93	0.93	539
weighted avg	0.93	0.93	0.93	539



AdaBoost Confusion Matrix



CNN – Convolutional NN

Neural Network:

- ❑ Kernel (3x3)
- ❑ Input (128x128)
- ❑ 4 hidden layers (ReLU)
- ❑ Output dense layer (SoftMax - 6 classes)

Data Augmentation: Random Flip

Dropout Rate

CNN – Hyperparameter Tuning

Keras_tuner:

- ❑ Conv Filter sizes: 32/64/128/256/512
- ❑ Dropout rate: 0.1-0.5
- ❑ Learning rate: $1e^{-2}$, $1e^{-3}$, $1e^{-4}$
- ❑ 20 Random Trials, early stopping
(patience = 3)

CNN – Batch Size Search

Trained the tuned model at 4 batch sizes

- ❑ Tested: [16, 32, 64, 128]
- ❑ Each training 20 epochs with early stopping

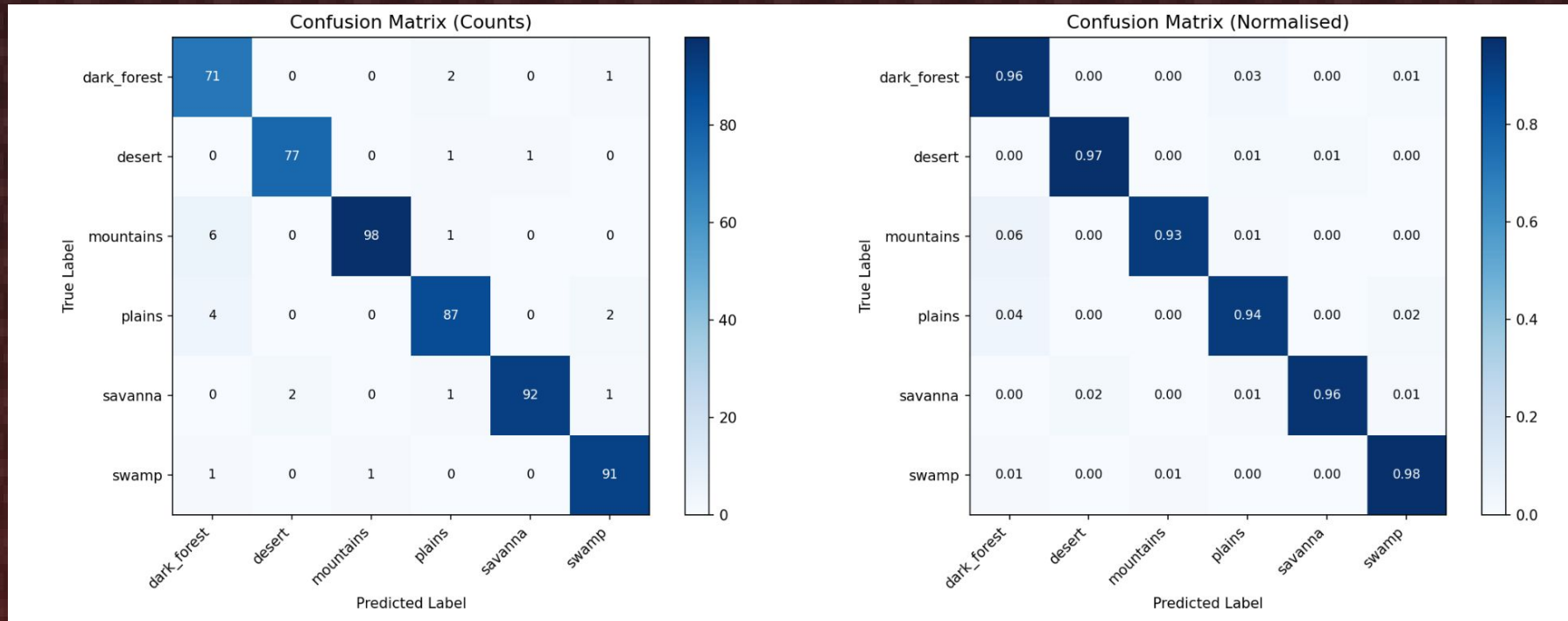
CNN – Best Configuration

- ❑ Best Hidden Node Configuration:

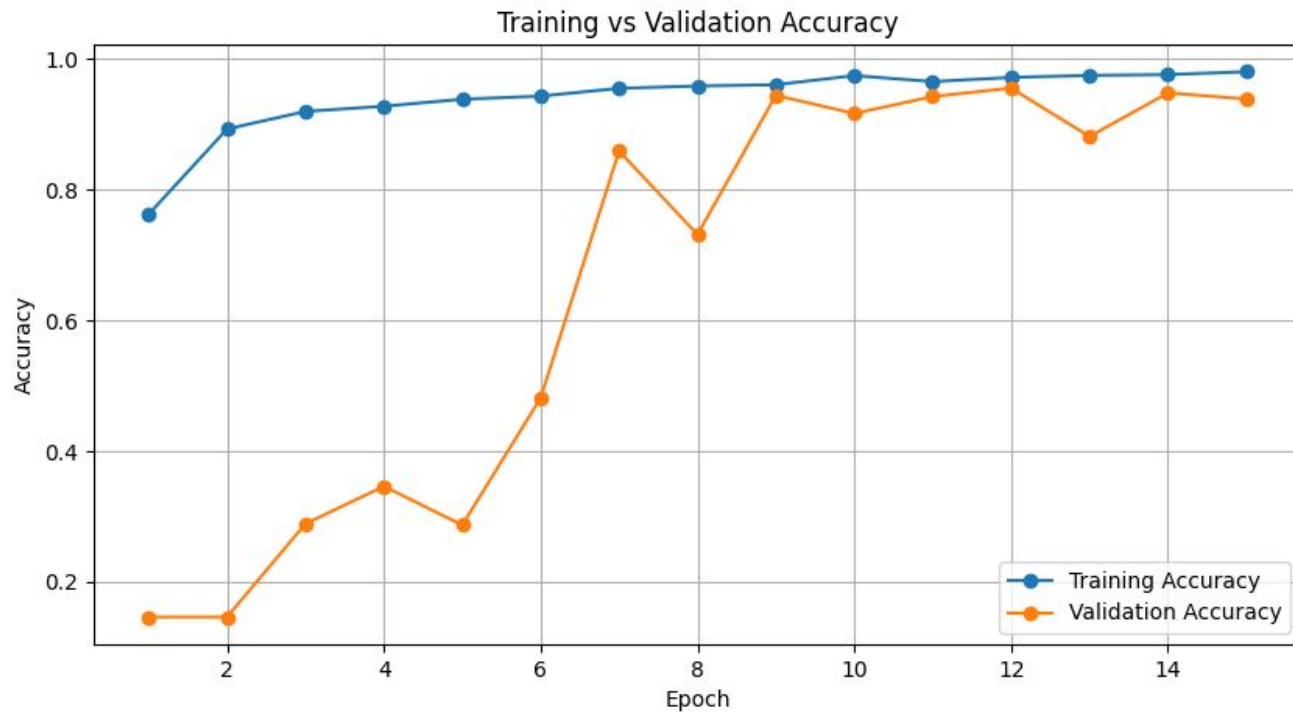
[128, 64, 256, 256, 256]

- ❑ Dropout Rate: 0.4 (pretty high)
- ❑ Best Learning Rate: $1.0 \cdot 10^{-4}$
- ❑ Best batch size: 32

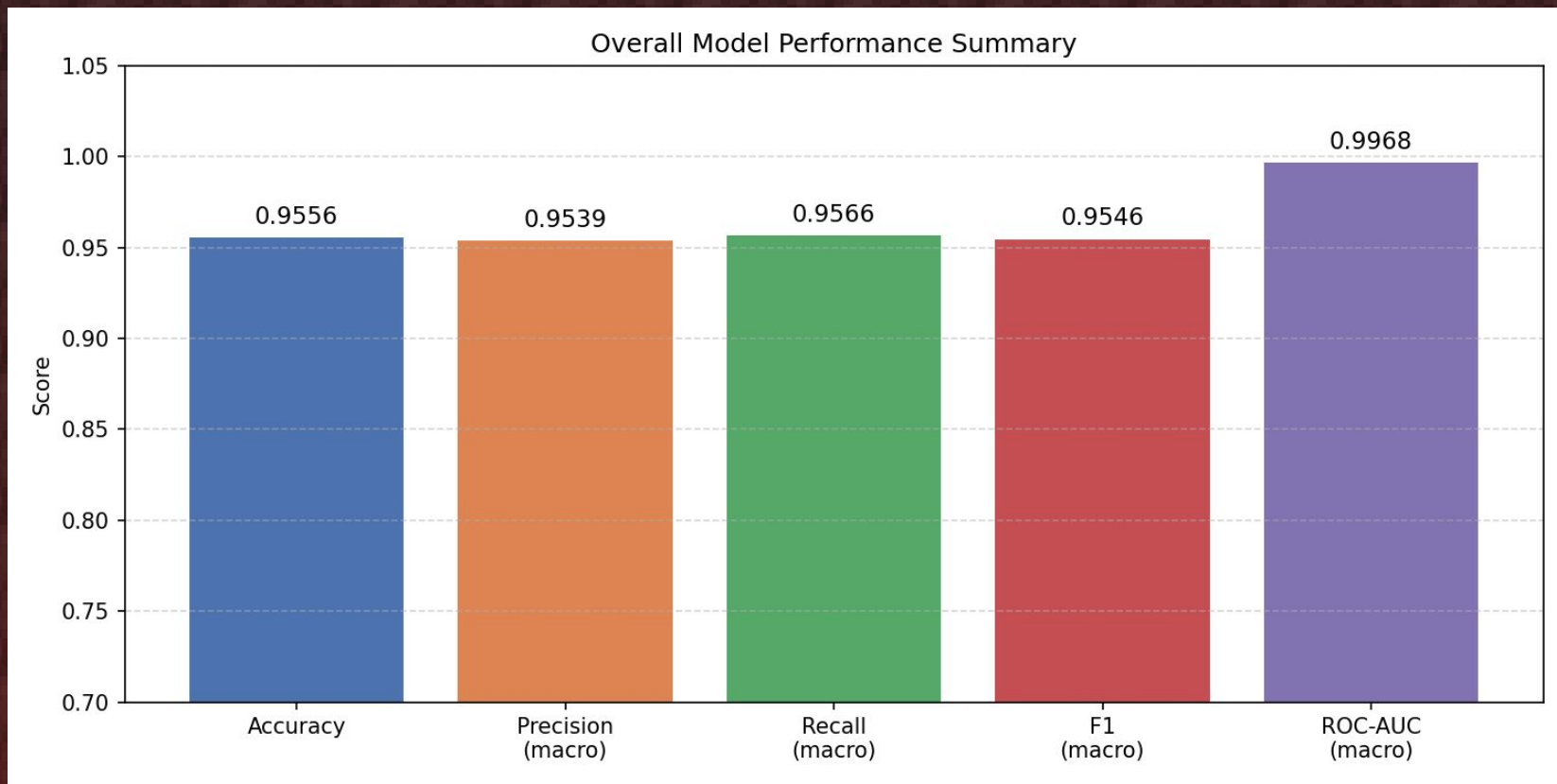
CNN-Results



CNN-Accuracy

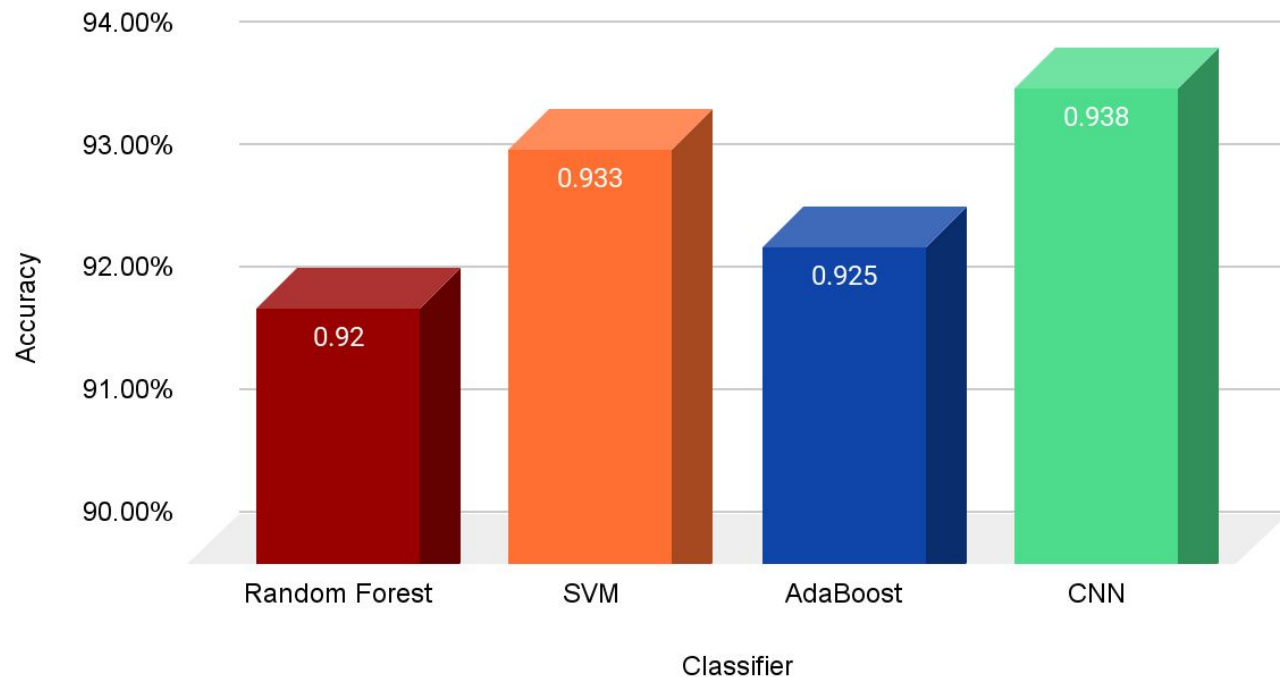


CNN-Summary



Comparison

Test Accuracy



Results

- Similar colors and textures led to some misclassification in all models
- CNN had the highest accuracy, however the results were very similar across models
- CNN was most computationally expensive to train



Limitations/ Future Works

- Increase the dataset
 - Currently we are only using 600 images per biome
- Similar textures and shapes led to misclassification
- Be able to identify if several biomes are visible in a single image.
- Add in more biomes with more similar features
- Use SMOTE to be able to use entirety of dataset
- Get GPUs

Acknowledgements

Dataset:

- <https://www.kaggle.com/datasets/willowc/minecraft-biomes/data>
 - willowc/minecraft-biomes

Preprocessing:

- <https://ernestsrudzitis.com/blog/color-meets-shape-using-histograms-of-gradients-and-colors-to-classify-flowers/>
- <https://www.geeksforgeeks.org/computer-vision/histogram-of-oriented-gradients/>